

AddLyft · Social Post Generation Report

Gemini 2.5 Flash-Lite on Vertex AI — how post generation is billed by tokens

A. Latest run (13 Sep 2026)

Fresh structured-output generation via `test_vertex_call.py` on Vertex AI. Brief: *20% off all yoga mats this weekend at ZenFit Studio.*

Field	Value
Mode	Text → structured social post (JSON schema)
Timestamp (UTC)	2026-09-13T13:54:08Z
Model	gemini-2.5-flash-lite
Path	Vertex AI (vertexai=True + ADC)
Prompt tokens	47
Output tokens	82
Thoughts tokens	0
Total tokens	129
List-price estimate	\$0.00003750
Log file	logs/usage.jsonl

Generated post:

Caption: Get ready to flow! This weekend only, enjoy a special discount on all yoga mats at ZenFit Studio.

Offer: 20% off all yoga mats

CTA: Visit us this weekend!

Hashtags: #ZenFitStudio #YogaMats #Sale #WeekendDeal #Yoga

B. Is the cost fixed per post?

No — cost is not a flat fee per post. Gemini on Vertex AI bills **by tokens consumed** on each successful call. Longer briefs, system instructions, JSON schema overhead, and longer captions all increase tokens (and therefore cost). Shorter replies cost less. The dollar amount for a given post is only known after the model returns `usage_metadata`.

What **is** fixed are the **unit rates** (price per million tokens) for the chosen model. Those rates stay the same; the bill = rates × tokens.

C. How billing depends on tokens

Each generation is split into token types. Flash-Lite rates used in this project's estimator (`usage.py`):

Token type	What it covers	Rate (USD / 1M)
Input (prompt)	Brief + system text + schema	\$0.10
Output	Generated JSON (caption, offer, CTA, tags)	\$0.40
Thoughts	Hidden reasoning, if any (billed as output)	\$0.40

Formula:

$$\text{estimated_usd} = (\text{prompt_tokens} \times 0.10 + (\text{output_tokens} + \text{thoughts_tokens}) \times 0.40) / 1,000,000$$

Latest run: $(47 \times 0.10 + 82 \times 0.40) / 1,000,000 = \text{\$0.0000375}$.

Tokens are counted by Gemini and returned in the API response — we do not count characters by hand. Charges accrue on the GCP project's linked billing account at Google's list rates for the model.

D. Logged post-generation runs

From `logs/usage.jsonl` (all `gemini-2.5-flash-lite`). Costs vary with token counts — same model, different bills:

When (UTC)	Source	P / O / Tot	Est. USD
2026-09-08 15:41	test_vertex_call	47 / 83 / 130	\$0.000038
2026-09-08 17:32	api_generate	109 / 116 / 225	\$0.000057
2026-09-08 17:32	api_generate	109 / 116 / 225	\$0.000057
2026-09-08 17:33	api_generate	109 / 102 / 211	\$0.000052
2026-09-08 17:35	api_generate	129 / 123 / 252	\$0.000062
2026-09-08 17:56	api_generate	117 / 132 / 249	\$0.000065
2026-09-08 18:19	api_generate	117 / 133 / 250	\$0.000065
2026-09-13 13:54	test_vertex_call	47 / 82 / 129	\$0.000038

Observed range for this app: about **\$0.000038** (short test brief) to about **\$0.000065** (API path with system instruction). Still far under one tenth of a cent per post. API calls use more prompt tokens because the system instruction is included.

E. Notes

- Tracking: `usage.py` + `backend/app/usage_log.py`; API returns usage on `/api/generate`.
- Rates can be overridden via `GEMINI_INPUT_PRICE_PER_MILLION` / `GEMINI_OUTPUT_PRICE_PER_MILLION`.
- Estimates use list rates; GCP Billing is the ledger of record.